

The Complete Vim Cheat Sheet

A progressive reference covering Vim from first keystrokes to scripting and automation.

Table of Contents

Notation Legend	5
Part 1: Basic and Everyday Use	5
Essential Navigation	6
Cursor Movement	6
Scrolling	6
Jumping to Lines	6
Basic Editing	7
Inserting Text	7
Deleting	7
Copy, Paste, Change	7
Undo / Redo	8
File Operations	8
Search Fundamentals	8
Visual Mode Basics	9
Part 2: Intermediate	9
Advanced Navigation	9
Marks	9
Jumps and Changes	9
Moving by Code Structure	10
Text Objects (navigation/editing combo)	10
Efficient Editing	10
Registers	10
Macros	11
Repeating Commands	11
Search and Replace	11

Basic Substitution	11
Useful Flags and Patterns	11
Window and Tab Management	12
Splitting Windows	12
Navigating Windows	12
Tabs	12
Buffers	12
Visual Mode Operations	13
Block Selection	13
Indenting and Formatting	13
Part 3: Advanced	13
Custom Configuration	13
vimrc Essentials	13
Key Mappings	14
Abbreviations	14
Advanced Search and Regex	14
Pattern Matching Basics	14
Very Magic Mode	14
Global Commands	15
Plugin Ecosystem Overview	15
Popular Plugin Managers	15
Essential Plugin Categories	15
Scripting and Automation	16
Vimscript Basics	16
Functions	16
Autocommands	16

Expert Workflows17

Sessions17

Diff Mode17

External Command Integration17

Where to Go Next17

A progressive reference covering Vim from first keystrokes to scripting and automation. Work through the parts in order if you're learning, or jump straight to the category you need if you're looking something up.

Notation Legend

Vim documentation (and this cheat sheet) uses a standard shorthand. Knowing it makes every command below unambiguous.

Notation	Meaning
<CR>	Press Enter / Return
<Esc>	Press Escape
<Tab>	Press Tab
<C-x>	Hold Ctrl and press x (e.g. <C-w> = Ctrl+w)
<S-x>	Hold Shift and press x
<leader>	The "leader" key, a user-defined prefix for custom mappings (default \, often remapped to , or <Space>)
[count]	An optional number typed before a command to repeat it (e.g. 5j moves down 5 lines)
{motion}	Any movement command (w, j, \$, gg, etc.) — many operators combine with a motion
{register}	A named storage slot, prefixed with " (e.g. "a is register a)
:	Enters Command-line mode (bottom of screen)
/ or ?	Enters search mode (forward/backward)
Mode names	Normal (default), Insert (typing text), Visual (selecting), Visual Block, Command-line, Replace

Operator + motion pattern: Most editing in Vim follows {operator}{count}{motion}, e.g. d2w (delete 2 words), ciw (change inner word), y\$ (yank to end of line). Learning the small set of operators (d, c, y, g~, >, <, =) and combining them with motions is what makes Vim fast — it's a grammar, not a list of shortcuts.

Part 1: Basic and Everyday Use

The commands you'll reach for in nearly every editing session. Master this part before moving on — everything later builds on it.

Essential Navigation

Cursor Movement

Command	Description
h j k l	Left, down, up, right
w	Jump to start of next word
b	Jump to start of previous word
e	Jump to end of current/next word
W B E	Same as above, but treat punctuation as part of the word (WORD vs word)
0	Jump to first column of the line
^	Jump to first non-blank character of the line
\$	Jump to end of the line
g_	Jump to last non-blank character of the line
f{char}	Find next occurrence of {char} on the line, cursor lands on it
F{char}	Find previous occurrence of {char} on the line
t{char}	Move until just before next {char}
T{char}	Move until just after previous {char} (searching backward)
;	Repeat last f/F/t/T
,	Repeat last f/F/t/T in opposite direction

Example: `dt,` deletes from the cursor up to (but not including) the next comma.

Scrolling

Command	Description
<C-d>	Scroll down half a screen
<C-u>	Scroll up half a screen
<C-f>	Scroll forward (down) a full screen
<C-b>	Scroll backward (up) a full screen
zz	Center the current line on screen
zt	Scroll current line to top of screen
zb	Scroll current line to bottom of screen

Jumping to Lines

Command	Description
gg	Go to first line of file
G	Go to last line of file
[count]G	Go to line [count] (e.g. 42G jumps to line 42)
: [count]<CR>	Same, typed in command mode (e.g. :42<CR>)
H	Move to top of visible screen
M	Move to middle of visible screen
L	Move to bottom of visible screen
%	Jump to matching bracket/paren/brace under cursor

Basic Editing

Inserting Text

Command	Description
<code>i</code>	Insert before cursor
<code>a</code>	Insert (append) after cursor
<code>I</code>	Insert at first non-blank of line
<code>A</code>	Insert (append) at end of line
<code>o</code>	Open new line below and insert
<code>O</code>	Open new line above and insert
<code><Esc></code>	Return to Normal mode from any insert/visual mode

Deleting

Command	Description
<code>x</code>	Delete character under cursor
<code>X</code>	Delete character before cursor
<code>dd</code>	Delete (cut) current line
<code>[count]dd</code>	Delete [count] lines (e.g. 3dd)
<code>dw</code>	Delete to start of next word
<code>d\$</code> or <code>D</code>	Delete to end of line
<code>d0</code>	Delete to start of line
<code>dG</code>	Delete to end of file
<code>dgg</code>	Delete to start of file

Copy, Paste, Change

Command	Description
<code>yy</code> or <code>Y</code>	Yank (copy) current line
<code>[count]yy</code>	Yank [count] lines
<code>p</code>	Paste after cursor / below line
<code>P</code>	Paste before cursor / above line
<code>cc</code>	Change (delete + insert) entire line
<code>cw</code>	Change to end of word
<code>r{char}</code>	Replace single character under cursor
<code>~</code>	Toggle case of character under cursor

Undo / Redo

Command	Description
<code>u</code>	Undo last change
<code><C-r></code>	Redo
<code>U</code>	Undo all latest changes on one line
<code>.</code>	Repeat last change (very useful — covered more in Intermediate)

File Operations

Command	Description
<code>:e {file}</code>	Open (edit) a file
<code>:w</code>	Save (write) current file
<code>:w {file}</code>	Save as a new filename
<code>:q</code>	Quit (fails if there are unsaved changes)
<code>:q!</code>	Quit and discard unsaved changes
<code>:wq</code> or <code>:x</code>	Save and quit
<code>:wq!</code>	Force save and quit
<code>ZZ</code>	Save and quit (Normal mode shortcut, same as <code>:wq</code>)
<code>ZQ</code>	Quit without saving (same as <code>:q!</code>)
<code>:saveas {file}</code>	Save current buffer under a new name and switch to it

Search Fundamentals

Command	Description
<code>/ {pattern} <CR></code>	Search forward for {pattern}
<code>? {pattern} <CR></code>	Search backward for {pattern}
<code>n</code>	Repeat last search, same direction
<code>N</code>	Repeat last search, opposite direction
<code>*</code>	Search forward for word under cursor
<code>#</code>	Search backward for word under cursor
<code>:noh</code>	Clear search highlighting

Example: `/error<CR>` finds the next "error"; pressing `n` repeatedly cycles through every match in the file.

Visual Mode Basics

Command	Description
<code>v</code>	Enter Visual mode (character-wise selection)
<code>V</code>	Enter Visual Line mode (whole-line selection)
<code><Esc></code>	Exit Visual mode
<code>{motion}</code> while in <code>v</code>	Extend selection using any movement command
<code>d</code>	Delete the selection
<code>y</code>	Yank the selection
<code>c</code>	Change the selection
<code>> / <</code>	Indent / un-indent selection

Example: `vjjd` selects two lines down from the cursor and deletes the selection.

Part 2: Intermediate

Once Basic feels automatic, these tools cut out repetitive work: multi-file navigation, recording your own commands, and precise find-and-replace.

Advanced Navigation

Marks

Command	Description
<code>m{a-z}</code>	Set a local mark named <code>{a-z}</code> at cursor position
<code>m{A-Z}</code>	Set a global mark (works across files)
<code>`{mark} ``</code>	Jump to exact position of <code>{mark}</code>
<code>'{mark}</code>	Jump to start of line containing <code>{mark}</code>
<code>` ``</code>	Jump back to position before last jump
<code>:marks</code>	List all current marks

Example: `ma` sets mark `a`; later, ``a ``` returns the cursor to that exact spot.

Jumps and Changes

Command	Description
<code><C-o></code>	Jump back to older position in jump list
<code><C-i></code>	Jump forward in jump list
<code>:jumps</code>	List the jump history
<code>g;</code>	Move to previous position in change list
<code>g,</code>	Move to next position in change list
<code>` . ``</code>	Jump to position of last edit

Moving by Code Structure

Command	Description
{	Jump to previous paragraph/blank-line block
}	Jump to next paragraph/blank-line block
()	Jump to previous/next sentence
[[]]	Jump to previous/next section (often function boundaries, language-dependent)
%	Jump between matching () { } [] (also matches #if/#endif etc. with config)
gd	Go to local declaration of variable under cursor
gD	Go to global declaration
]p	Paste with indentation matched to current line

Text Objects (navigation/editing combo)

Command	Description
iw / aw	Inner word / a word (with surrounding space)
ip / ap	Inner paragraph / a paragraph
i" / a"	Inner quoted string / a quoted string
i(or ib	Inside parentheses
i{ or iB	Inside braces
it / at	Inner tag / a tag (HTML/XML)

Example: `ci"` changes the text inside the nearest quotes, regardless of cursor position within them.

Efficient Editing

Registers

Command	Description
"{reg}yy	Yank line into named register {reg} (a-z)
"{reg}p	Paste from named register {reg}
"{reg}dd	Delete line into named register {reg}
" +y	Yank into the system clipboard
" +p	Paste from the system clipboard
"0p	Paste from the yank register (last yank, unaffected by deletes)
:reg	List contents of all registers
"_d	Delete into the "black hole" register (doesn't overwrite last yank)

Example: `"ayy` then later `"ap` lets you yank a line into register a and paste it elsewhere without disturbing your default unnamed register.

Macros

Command	Description
q{a-z}	Start recording a macro into register {a-z}
q	Stop recording
@{a-z}	Play back macro {a-z}
@@	Repeat the last played macro
[count]{a-z}	Play macro [count] times

Example: qaA;<Esc>q records a macro (in register a) that appends a semicolon to the end of a line; 10@a then runs it on the next 10 lines.

Repeating Commands

Command	Description
.	Repeat last change exactly
[count].	Repeat last change, overriding its count
:g/{pattern}/normal {cmd}	Run a Normal-mode command on every matching line (preview of global command, see Advanced)
&	Repeat last :s substitution on current line

Search and Replace

Basic Substitution

Command	Description
:s/old/new/	Replace first match of old with new on current line
:s/old/new/g	Replace all matches on current line
:%s/old/new/g	Replace all matches in the whole file
:%s/old/new/gc	Replace all matches, confirming each one
:'<,'>s/old/new/g	Replace within a Visual selection (auto-filled range after selecting then typing :)
:5,10s/old/new/g	Replace within line range 5-10

Useful Flags and Patterns

Flag/Pattern	Description
g	Replace all occurrences in the line, not just the first
c	Confirm each substitution (y/n/a/q)
i	Case-insensitive match for this command
\1, \2, ...	Backreferences to captured groups \(\. . .\) in replacement text
& (in replacement)	Refers to the whole match

Example: :%s/\(\\w\\+\)@\\(\\w\\+\)/\2@\\1/g swaps text on either side of an @ symbol across the whole file.

Window and Tab Management

Splitting Windows

Command	Description
<code>:split</code> or <code>:sp</code>	Split window horizontally
<code>:vsplit</code> or <code>:vsp</code>	Split window vertically
<code>:sp {file}</code>	Split and open {file} in the new window
<code><C-W>s</code>	Horizontal split (Normal mode shortcut)
<code><C-W>v</code>	Vertical split (Normal mode shortcut)
<code><C-W>q</code>	Close current window
<code><C-W>o</code>	Close all windows except current

Navigating Windows

Command	Description	
<code><C-W>w</code>	Cycle to next window	
<code><C-W>h/j/k/l</code>	Move to window left/down/up/right	
<code><C-W>=</code>	Equalize all window sizes	
<code><C-W>_</code>	Maximize current window's height	
<code>`<C-W></code>	`	Maximize current window's width

Tabs

Command	Description
<code>:tabnew {file}</code>	Open {file} in a new tab
<code>gt</code>	Go to next tab
<code>gT</code>	Go to previous tab
<code>[count]gt</code>	Go to tab number [count]
<code>:tabclose</code>	Close current tab
<code>:tabs</code>	List all open tabs

Buffers

Command	Description
<code>:ls</code> or <code>:buffers</code>	List all open buffers
<code>:b {name/number}</code>	Switch to a buffer by partial name or number
<code>:bnext</code> / <code>:bn</code>	Go to next buffer
<code>:bprev</code> / <code>:bp</code>	Go to previous buffer
<code>:bd</code>	Delete (close) current buffer

Visual Mode Operations

Block Selection

Command	Description
<code><C-v></code>	Enter Visual Block mode (select a rectangular column)
<code>I</code> (in block mode)	Insert text at the start of every selected line at once
<code>A</code> (in block mode)	Append text at the end of every selected line at once
<code>c</code> (in block mode)	Change the block across all selected lines
<code>\$</code> (in block mode)	Extend block to end of each line (handles ragged-right text)

Example: Select a column of 5 lines with `<C-v>4j`, press `I//`, then `<Esc>` to comment out all 5 lines at once.

Indenting and Formatting

Command	Description
<code>> / <</code>	Indent / un-indent selection by one shiftwidth
<code>gv</code>	Reselect the last Visual selection
<code>=</code>	Auto-indent selection using filetype rules
<code>gu</code>	Lowercase selection
<code>gU</code>	Uppercase selection
<code>g~</code>	Toggle case of selection
<code>J</code> (Visual)	Join selected lines into one

Part 3: Advanced

This is where Vim becomes a programmable editor: configuration, regex mastery, plugins, scripting, and workflows for serious development.

Custom Configuration

vimrc Essentials

Your configuration lives at `~/.vimrc` (or `~/.config/nvim/init.vim` for Neovim). A few foundational settings:

```
" General behavior
set nocompatible
set number
set relativenumber
set hlsearch
set incsearch
set ignorecase
set smartcase
set expandtab
set tabstop=4
set shiftwidth=4
set autoindent
set wrap
set scrolloff=8
syntax on
filetype plugin indent on

" Use Vim defaults, not vi-compatible mode
" Show line numbers
" Show relative line numbers (great for counts/motions)
" Highlight search matches
" Show matches as you type
" Case-insensitive search...
" ...unless the pattern has an uppercase letter
" Use spaces instead of tabs
" Width of a tab character
" Width used by indent operations
" Copy indent from current line on new line
" Visually wrap long lines
" Keep 8 lines visible above/below cursor
" Enable syntax highlighting
" Enable filetype-specific plugins/indenting
```

Key Mappings

Command	Description
<code>:map</code>	Show all mappings (all modes)
<code>:nmap {lhs} {rhs}</code>	Map in Normal mode (recursive — can trigger other mappings)
<code>:nnoremap {lhs} {rhs}</code>	Map in Normal mode (non-recursive — safer, preferred)
<code>:inoremap {lhs} {rhs}</code>	Map in Insert mode
<code>:vnoremap {lhs} {rhs}</code>	Map in Visual mode
<code>let mapleader = ","</code>	Change the leader key from default \

```
" Example mappings in .vimrc
nnoremap <leader>w :w<CR>          " <leader>w saves the file
nnoremap <C-h> <C-w>h              " Faster window navigation
inoremap jk <Esc>                  " Use jk to exit Insert mode
nnoremap <leader>s :%s/\<<C-r><C-w>\> //g<Left><Left> " Quick rename word under
cursor
```

Tip: Always prefer the `noremap` variants in your config — plain `map/nmap` can cause unexpected recursive behavior if mappings reference each other.

Abbreviations

Command	Description
<code>:ab {short} {full}</code>	Expand {short} into {full} automatically when typed (Insert/Command mode)
<code>:iab {short} {full}</code>	Same, but Insert mode only
<code>:una {short}</code>	Remove an abbreviation

```
iab teh the
iab @@ myemail@example.com
iab sig Best regards,<CR>Jane Doe
```

Advanced Search and Regex

Pattern Matching Basics

Pattern	Meaning	
<code>.</code>	Any single character	
<code>*</code>	Zero or more of the previous atom	
<code>\+</code>	One or more (regular "magic" mode)	
<code>\?</code>	Zero or one	
<code>^</code>	Start of line	
<code>\$</code>	End of line	
<code>\< \></code>	Word boundaries (start/end)	
<code>[abc]</code>	Character class: a, b, or c	
<code>[^abc]</code>	Negated class: anything except a, b, c	
<code>\(...\)</code>	Capture group (regular magic mode)	
<code>\</code>	<code>`</code>	Alternation (OR)

Very Magic Mode

Prefixing a pattern with `\v` switches to "very magic" mode, where most special characters (`+`, `?`, `(`, `)`, `|`, `{`, `}`) work without backslashes — much closer to Perl/PCRE regex syntax.

Mode	Example	Equivalent	
Magic (default)	/foo\+bar	one-or-more "o" before "bar"	
Very magic	/\vfoo+bar	same pattern, cleaner syntax	
Very magic groups	`\v(foo	bar)+`	no backslashes needed for groups/alternation

Example: `/\v^(import|from)\s+\w+` finds Python import lines without backslash clutter.

Global Commands

Command	Description
<code>:g/{pattern}/{cmd}</code>	Run {cmd} on every line matching {pattern}
<code>:g/{pattern}/d</code>	Delete every line matching {pattern}
<code>:g!/{pattern}/d</code> or <code>:v/{pattern}/d</code>	Delete every line not matching {pattern}
<code>:g/{pattern}/normal {keys}</code>	Run Normal-mode {keys} on every matching line
<code>:g/{pattern}/m0</code>	Move every matching line to the top (also a quick reverse/sort trick)
<code>:g/{pattern}/s//{repl}/g</code>	Substitute only within matching lines

Example: `:g/TODO/normal A — pending<CR>` appends " — pending" to the end of every line containing "TODO".

Plugin Ecosystem Overview

Popular Plugin Managers

Manager	Notes
vim-plug	Lightweight, single-file install, widely used, simple <code>Plug 'user/repo'</code> syntax
packer.nvim	Neovim-focused, Lua-based, supports lazy-loading
lazy.nvim	Modern Neovim standard, fast startup via deferred loading
Vundle	One of the original plugin managers, still functional but less active
native packages	Vim 8+/Neovim built-in <code>packadd</code> , no external manager required

Essential Plugin Categories

Category	Purpose	Well-known examples
Fuzzy finder	Quickly open files, search content, jump to symbols	fzf.vim, telescope.nvim
File explorer	Tree-style file browser sidebar	NERDTree, nvim-tree
Status line	Richer bottom status bar (mode, git branch, errors)	vim-airline, lualine.nvim
Git integration	Diff gutter, blame, staging from inside Vim	vim-fugitive, gitsigns.nvim
Linting/LSP	Real-time error checking, autocompletion, go-to-definition	coc.nvim, native LSP (Neovim)
Surround text	Add/change/delete surrounding quotes, brackets, tags	vim-surround
Commenting	Toggle line/block comments	vim-commentary, Comment.nvim
Auto-pairs	Auto-close brackets/quotes	auto-pairs

Plugins are typically installed by adding a line to your config and running an install command (e.g. `:PlugInstall` for vim-plug), then restarting Vim.

Scripting and Automation

Vimscript Basics

```
" Variables
let g:my_var = "hello"      " Global variable
let s:my_var = "world"      " Script-local variable
let b:my_var = "buffer"     " Buffer-local variable

" Conditionals
if &filetype ==# 'python'
  set shiftwidth=4
elseif &filetype ==# 'javascript'
  set shiftwidth=2
endif

" Loops
for i in range(1, 5)
  echo i
endfor
```

Functions

```
function! GreetUser(name)
  echo "Hello, " . a:name . "!"
endfunction

call GreetUser("World")

" Function returning a value
function! DoubleNumber(n)
  return a:n * 2
endfunction

echo DoubleNumber(21)      " => 42
```

Element	Meaning
<code>function!</code>	The <code>!</code> allows redefining the function without error
<code>a:name</code>	Access an argument passed to the function
<code>g:, s:, b:, l:</code>	Scope prefixes: global, script-local, buffer-local, function-local
<code>:call {Func}()</code>	Invoke a function

Autocommands

Autocommands run code automatically when specific events occur (opening a file, saving, changing filetype, etc.).

```
" Trim trailing whitespace on save
autocmd BufWritePre * :%s/\s\+$/e

" Set 2-space indent for web files
autocmd FileType html,css,javascript setlocal shiftwidth=2 tabstop=2

" Group autocommands to avoid duplicate registration on reload
augroup MyAutocommands
  autocmd!
  autocmd BufEnter *.md setlocal spell
augroup END
```

Common Event	Fires when...
<code>BufWritePre</code>	Just before a buffer is saved
<code>BufWritePost</code>	Just after a buffer is saved
<code>BufEnter</code>	A buffer becomes the active window

Common Event	Fires when...
FileType	Filetype is detected/set
InsertLeave	Leaving Insert mode
VimEnter	Vim has finished starting up

Expert Workflows

Sessions

Command	Description
<code>:mksession {file}.vim</code>	Save current layout (windows, tabs, buffers, cursor positions) to a session file
<code>vim -S {file}.vim</code>	Launch Vim restoring that session
<code>:source {file}.vim</code>	Load a session into the current Vim instance

Useful for resuming a multi-file, multi-split project exactly where you left off.

Diff Mode

Command	Description
<code>vim -d {file1} {file2}</code>	Open two files side-by-side in diff mode
<code>:diffsplit {file}</code>	Diff current file against {file} in a new split
<code>]c</code>	Jump to next difference
<code>[c</code>	Jump to previous difference
<code>do</code>	Obtain (pull in) change from the other file
<code>dp</code>	Put (push) change to the other file
<code>:diffupdate</code>	Recalculate diff highlighting

External Command Integration

Command	Description
<code>:!{cmd}</code>	Run a shell command without leaving Vim
<code>:r !{cmd}</code>	Run a shell command and insert its output into the buffer
<code>:%!{cmd}</code>	Filter the entire buffer through an external command (e.g. <code>:%!sort</code> , <code>:%!python3 -m json.tool</code>)
<code>'<, '>!{cmd}</code>	Filter a Visual selection through an external command
<code><C-z> then fg</code>	Suspend Vim to the shell, return later with <code>fg</code>
<code>:terminal</code>	Open an interactive terminal inside Vim (Vim 8+/Neovim)

Example: Select a block of unsorted lines in Visual mode, then run `:'<, '>!sort` to pipe just that selection through the `sort` command and replace it with sorted output.

Where to Go Next

Once these three parts feel comfortable, the natural next steps are: reading `:help` directly inside Vim (it's thorough and cross-referenced), building a personal `.vimrc` incrementally rather than copying a large one wholesale, and picking one or two plugins from each category above rather than installing everything at once.